THE POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY 1

EMBEDDED SYSTEMS REPORT

ENDER SOCKET — IoT Smart Socket

GROUP : 7

MEMBERS: NGUYỄN TUẤN MINH - B22DCKH077

NGUYỄN XUÂN KIÊN - B22DCKH062

NGUYỄN VIỆT HOÀNG - B22DCVT214

LÊ ĐỨC MẠNH - B22DCAT184

Hà Nội - 2026

MENU

1. Problem Definition and System Requirements..... 2
1.1. Real-World problem description..... 2
1.2. Functional requirements..... 3
1.3. Non-functional requirements.....4
2. System design..... 8
2.1. Overall architecture..... 8
2.2. Hardware design.....10
2.3. Software design..... 12
3. Deploy & Coding.....18
3.1. Firmware for ESP32..... 18
3.2. Code structure(Modular Design).....28
4. Evaluation and Testing..... 28
5. Innovation & Scalability.....32
5.1. Technical innovations..... 32
5.2. Future scalability..... 33
6. Conclusion..... 34

1. Problem Definition and System Requirements

1.1. Real-World problem description

Context and Inherent Risks

In modern environments, users frequently charge high-capacity lithium batteries—such as those used in e-bikes, laptops, or large power banks—by leaving them plugged in overnight. While this habit is highly convenient, it introduces two critical safety hazards:

- **Overcharging:** When a lithium battery is continuously forced to charge beyond its 100% capacity threshold over an extended period, its internal chemical structure degrades. This not only drastically reduces the battery's lifespan but also creates a significant risk of fire or explosion.
- **Uncontrolled Heat Generation:** Most standard chargers and electrical sockets currently on the market lack integrated temperature monitoring. If technical issues arise—such as a localized short circuit or poor electrical contact generating excess heat—a fire could easily ignite while the user is asleep, with no preventive systems in place to intervene.

Target Use Case

A user plugs in an e-bike battery (specifications: 48V, approximately 3A charging current) at night and requires a completely automated system to manage the process.

In this scenario, the Ender Socket replaces the standard electrical socket. Its primary function is to continuously measure the charging current and strictly monitor the temperatures at both the plug and the Solid State Relay (SSR). The device will automatically cut off the power supply the moment its algorithm detects that the battery is fully charged. Furthermore, it will immediately dispatch alert notifications to the user if the hardware temperature exceeds established safety thresholds. This entire operational cycle is a closed-loop system that requires zero human intervention.

Practical Value of the Project

- **Energy Efficiency:** Reduces power consumption through an active mechanism that cuts electricity exactly when the battery reaches its maximum capacity.
- **Battery Protection:** Shields battery cells from continuous overcharging, thereby maximizing the hardware's overall lifespan.
- **Fire Prevention:** Acts as a vital safety checkpoint against fires and explosions via continuous, real-time temperature monitoring.

- **Remote Accessibility:** Grants users the ability to control, manage, and monitor system parameters from anywhere via an Internet connection.

1.2. Functional requirements

ID	Requirement	Description
FR-01	Power Measurement	Measure voltage (V), current (A), power (W), and accumulated energy (Wh) in real-time.
FR-02	Temperature Monitoring	Measure the temperature at 2 locations: the plug (Socket) and the SSR controller.
FR-03	Relay Control	Turn on/off the load power via the SSR relay locally (physical button) and remotely (MQTT).
FR-04	Smart Charging	Automatically detect a full battery, an unplugged device, or abnormal current, and cut off the relay accordingly.
FR-05	Safety Alerts	Issue a warning and cut off the relay when the temperature exceeds 60°C; permanently lock the relay when the temperature exceeds 75°C.
FR-06	Information Display	Display all measured parameters on a 320×240 TFT LCD screen.
FR-07	MQTT Connection	Send telemetry data and receive control commands over the Internet via the MQTT protocol.
FR-08	Web Dashboard	Provide a web interface to remotely monitor and control the device.
FR-09	WiFi Configuration	Configure the WiFi network without needing to reprogram the device via a Captive Portal.

FR-10	Physical Control Buttons	BTN1: Reset Wh / Toggle Smart Mode; BTN2: Turn screen on/off / Open WiFi Portal.

1.3. Non-functional requirements

a) Real-time constraints

Tasks are separated by strict real-time priorities utilizing a FreeRTOS Dual-Core architecture:

Core 1 (Safety-Critical Domain) Requirements:

#	Task	Threshold	Design Rationale
1	Safety Monitoring Cycle	50 ms	To guarantee a rigid, uninterrupted execution interval for critical operations, ensuring that safety monitoring is completely isolated from and unaffected by unpredictable network delays or reconnections.
2	Read Power Meter	1,000 ms	To provide a steady, 1Hz update rate that accurately captures electrical load variations while maintaining synchronization with the system's standard telemetry broadcasting interval.
3	Read Temperature Sensors	800 ms	To accommodate the inherent hardware conversion time required by the temperature sensors, allowing the system to handle this process asynchronously without blocking other critical safety checks.

4	Evaluate Safety Conditions	50 ms	To ensure highly responsive, near-instantaneous detection of critical events—such as sudden current drops or dangerous temperature spikes—allowing the system to trigger an immediate hardware cutoff before hazards escalate.
5	Update Shared System State	50 ms	To securely synchronize the latest power readings and safety statuses with the secondary core, ensuring the user interface and remote dashboards reflect real-time data without compromising the strict isolation between safety and communication tasks.

Core 0 (Communication & UI Domain) Requirements:

#	Task	Threshold	Design Rationale
1	Yield CPU Execution	10 ms	To consistently release processing time, preventing the communication loop from monopolizing the core and allowing background networking stacks to execute smoothly.
2	Update LCD Screen	500 ms	To provide a responsive user interface with the latest data without causing visual flicker or wasting system resources on imperceptible screen redraws.
3	Send Telemetry Data	1,000 ms	To maintain a steady, real-time data stream to the remote dashboard while conserving network bandwidth and preventing server overload.

4	Debounce Physical Buttons	50 ms	To filter out rapid mechanical electrical noise, ensuring that a single physical press is reliably registered as exactly one user input.
5	Verify Command Execution	150 ms	To provide a brief physical delay that allows hardware components (like the mechanical relay) sufficient time to actuate before verifying their true state and sending an acknowledgment to the user, thereby preventing "ghost" UI states.
6	WiFi Reconnection Retry	5,000 ms	To allow the networking hardware adequate time to attempt a complete connection handshake without continuously blocking other tasks on the communication core.
7	Cloud Reconnection Retry	3,000 ms	To swiftly restore remote telemetry and control capabilities following a network drop, balancing rapid recovery with system stability.
8	Evaluate Offline Status (Frontend/Dashboard)	5,000 ms	To actively monitor the freshness of incoming data on the user's dashboard, ensuring the interface can reliably detect and alert the user if the device stops broadcasting for any reason.

Architectural Note on Core Placement: These tasks are intentionally placed on a low-priority secondary core. Because operations like network reconnections can experience unpredictable delays, isolating them guarantees that the critical safety monitoring (running on the primary high-priority core) is never delayed or compromised by communication failures.

b) Resource constraints

Component	Estimated Price
ESP32 Dev Module	~80,000 VND
ILI9341 2.4" TFT Display	~60,000 VND
PZEM-004T v3.0	~120,000 VND
2× DS18B20 Sensors	~40,000 VND
SSR Fotek 25DA	~90,000 VND
2× TTP223 (touch modules)	~20,000 VND
LEDs, buzzer, resistors, power supply	~30,000 VND
Total	~440,000 VND

c) Reliability & Safety requirements

- **Permanent Fire Lock:** Upon detecting critical fire-risk temperatures, the system triggers an irreversible lock mechanism. The power supply is permanently disabled and cannot be overridden by any software or remote commands. Normal operation can only be restored by physically disconnecting and reconnecting the device's main power supply.
- **Exclusive Hardware Control:** The physical manipulation of the power relay is strictly restricted to a single, secure authorization pathway within the safety-critical processing domain. No secondary processes, user interfaces, or networking tasks are permitted direct access to the hardware switch.
- **Network-Independent Safety Monitoring:** The safety observation routines operate completely autonomously from the communication systems. Should the

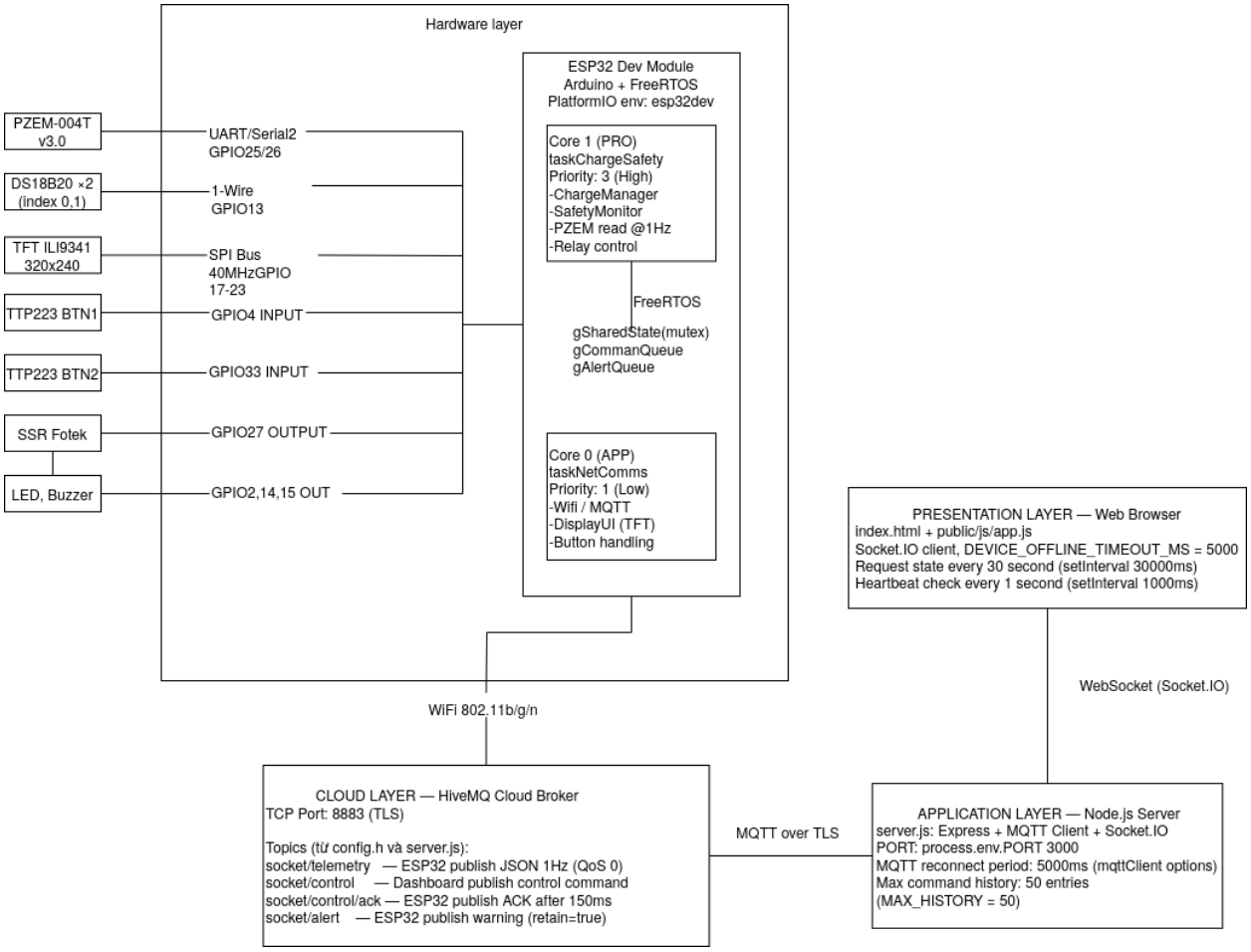
device lose network connectivity, the system continues its strict, uninterrupted monitoring of power consumption and temperature levels, retaining the full capability to sever power regardless of background reconnection attempts.

- **Strict Processing Isolation:** The system architecture permanently assigns all critical safety and environmental monitoring tasks to a dedicated processing core operating at the highest priority. All networking, background communications, and user interface tasks are confined to a secondary core, structurally guaranteeing that critical safety checks are never delayed, preempted, or interrupted by network operations.

2. System design

2.1. Overall architecture

2.1.1. Block diagram



2.1.2. Embedded software layer - FreeRTOS Dual-Core

```
ORCHESTRATION — setup() + FreeRTOS Scheduler
setup():
    ├─ initGPIO (all pin before start task)
    ├─ WiFiStorage::initSPIFS()
    ├─ gStateMutex   = xSemaphoreCreateMutex()
    ├─ gCommandQueue = xQueueCreate(6,  sizeof(RelayCommand))
    ├─ gAlertQueue   = xQueueCreate(12, sizeof(AlertMessage))
    ├─ xTaskCreatePinnedToCore(taskChargeSafety, Core 1, prio=3)
    └─ xTaskCreatePinnedToCore(taskNetComms,     Core 0, prio=1)
loop() → vTaskDelete(nullptr)  [auto delete, unused]
```

```
CORE 1: taskChargeSafety
(Priority 3 — HIGH)
vTaskDelayUntil() 50ms
├─ xQueueReceive()
│   CmdType::RELAY
│   CmdType::SMART_MODE
│   CmdType::RESET_ENERGY
├─ readPzem() @1Hz
├─ safety.update()
├─ handleSafetyLogic()
│   ├─ overheat relay OFF
│   └─ fireLocked = true
├─ charger.update()
├─ detect spike alert
└─ xSemaphoreTake(mutex)
write gSharedState
xSemaphoreGive(mutex)
```

```
CORE 0: taskNetComms
(Priority 1 — LOW)
vTaskDelay() 10ms
├─ BTN1 / BTN2 polling + FSM
├─ connectWifiIfNeeded()
├─ connectMqttIfNeeded()
├─ gMqtt.loop()
│   └─ mqttCallback()
│       └─ xQueueSend(cmd)
├─ xQueueReceive(alertQueue)
├─ netPublishAlert() MQTT
├─ processPendingAck()
├─ SharedState snapshot
├─ gUi.update(snap)
└─ publishTelemetry(snap)
xSemaphoreTake(mutex)
read gSharedState
xSemaphoreGive(mutex)
```

```
Domain module

ChargeManager (Core 1)
├─ commandRelay()
├─ forceRelayOff()
├─ setSmartMode()
├─ update()
├─ _pushCurrent()
├─ _rollingAvg()
├─ _setRelay() GPIO27
└─ _resetSession()

SafetyMonitor (Core 1)
├─ begin() — DS18B20 init
├─ update() — async 2-phase
├─ status() — SafetyStatus
├├─ setPeriphFault()
├─ _updateLedsAndBuzzer()
├─ _isValidTemp()

SharedState (inter-core)
├─ PowerReading
├─ SafetyStatus
├─ relayOn, smartMode
└─ fireLocked, lastAlert

DisplayUI (Core 0 ONLY)
├─ begin() — TFT init (SPI)
├─ update(SharedState snap)
├─ toggleDisplay() GPIO32
├─ _drawStatic() — 1 lần
├─ _drawField() — partial update
└─ _stopReasonStr()
```

```
HARDWARE ABSTRACTION — Libraries (từ platformio.ini)

bodmer/TFT_eSPI @ ^2.5.43          — ILI9341 SPI driver
mandulaj/PZEM-004T-v30 @ ^1.1.2    — Power meter UART driver
paulstoffregen/OneWire @ ^2.3.8    — 1-Wire bus driver
milesburton/DallasTemperature @ ^4.0.5 — DS18B20 driver
knolleary/PubSubClient @ ^2.8      — MQTT client
tzapu/WiFiManager @ ^2.0.17        — Captive Portal
bblanchon/ArduinoJson @ ^7.0.0     — JSON serialization
```

```
HARDWARE  (GPIO, UART, SPI, 1-Wire)
```

2.2. Hardware design

2.2.1. Reason for choosing ESP32 (MCU comparison)

Tiêu chí	Arduino UNO	STM32F103	ESP32
CPU	16 MHz AVR	72 MHz ARM M3	240 MHz Xtensa LX6
RAM	2 KB	20 KB	520 KB
WiFi tích hợp	No	No	802.11 b/g/n
Bluetooth	No	No	BT 4.2 + BLE
UART hardware	1	3	3 (using Serial2 for PZEM)
SPI hardware	1	2	4
Giá	~80K VND	~120K VND	~100K VND
GPIO count	14	37	34

Conclusion for choosing ESP32: Come with integrated WiFi (MQTT connection), has enough RAM for TCP/TLS (~50KB) and application stack, has UART for PZEM-004T, has high speed SPI for 40MHz screen. Arduino UNO and STM32 can not satisfy the requirements in the same price range.

2.2.2. GPIO layout

GPIO	Module	Connection	Role	Core
18	SPI Bus	SPI CLK	Clock for TFT + Touch	0
19	SPI Bus	SPI MISO	Data from TFT/Touch to ESP32	0

23	SPI Bus	SPI MOSI	Data from ESP32 to TFT	0
17	ILI9341 CS	SPI CS	Chip Select for screen	0
16	ILI9341 DC	Digital	Data/Command select	0
5	ILI9341 RST	Digital	Hardware reset for screen	0
32	Backlight	Digital OUT	LCD light control	0
21	XPT2046	SPI CS	Touch chip select	0
12	SD Card	SPI CS	Unused	-
26	PZEM-004T	UART RX	Receive data from power meter	1
25	PZEM-004T	UART TX	Send commend to power meter	1
13	DS18B20 ×2	1-Wire	Bus for dual temperature sensor	1
4	TTP223 BTN1	Digital IN	Main touch button	0
33	TTP223 BTN2	Digital IN	Secondary touch button	0
27	SSR Fotek	Digital OUT	Open/close AC module	1
14	Green LED	Digital OUT	Wifi state	0
2	Red LED	Digital OUT	Error/Overheat/Fire	1
15	Buzzer	Digital OUT	Fire risk	1

2.2.3. Module choosing

PZEM-004T v3.0: A specialized AC power measurement module utilizing Modbus-RTU communication via UART at 9600bps. The ‘mandulaj/PZEM-004T-v30’ library provides a simple API for data retrieval (e.g., pzem.voltage(), pzem.current(), pzem.power(), and

pzem.energy()). It features electrical isolation for safety, eliminating the need for dangerous, custom-designed AC current sensing circuits.

DS18B20: A 1-Wire temperature sensor. Two sensors are connected to the same bus on GPIO 13 and are distinguished by their index (index(0) for the socket, index(1) for the SSR). This configuration saves GPIO pins. Using setWaitForConversion(false) enables non-blocking reads, which is critical for efficient software design.

SSR Fotek 25DA: A Solid State Relay controlled by 3–32VDC, making it directly compatible with the 3.3V GPIO of the ESP32. Logic levels are defined in config.h as RELAY_ON = HIGH and RELAY_OFF = LOW.

TTP223: A capacitive touch sensor that outputs HIGH when touched; interfaced easily via a standard digitalRead().

2.3. Software design

2.3.1. Scheduling Model: FreeRTOS Preemptive Dual-Core

Instead of 'loop()' cooperative scheduling, firmware uses FreeRTOS Preemptive multi-tasking with 2 tasks pinned into 2 cores:

Core 1 (PRO Core)

taskChargeSafety [prio=3]

vTaskDelayUntil(&wake, 50ms) ← ensure hard cycle 50ms

[0ms] drain gCommandQueue

[1ms] pzem.readAll() if @1Hz

[2ms] safety.update()

[3ms] handleSafety (fire/overheat)

[4ms] charger.update()

[5ms] write gSharedState (mutex)

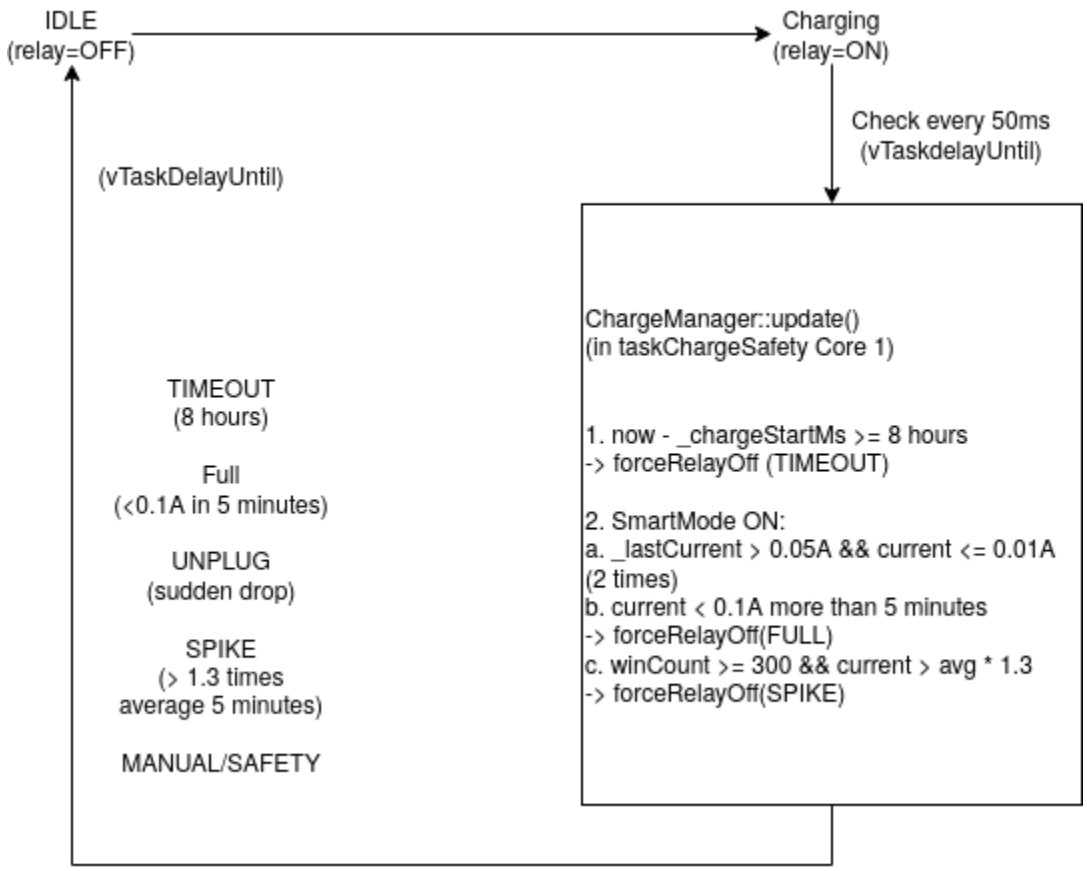
[vTaskDelayUntil: time to sleep]

Core 0 (APP Core)
taskNetComms [prio=1]

vTaskDelay(10ms)← yield, non-blocking

[0ms] BTN1/BTN2 FSM
[2ms] WiFi reconnect
[3ms] MQTT.loop()
[4ms] drain gAlertQueue
[5ms] ACK verify
[6ms] read gSharedState (mutex)
[7ms] gUi.update(snap)
[8ms] publishTelemetry
[vTaskDelay: time to sleep]

2.3.2. Finite state machine



2.3.3. FSM for BTN1 and BTN2

Run on Core 0 in `taskNetComms`. Instead of directly calling `charger.commandRelay()` or`charger.setSmartMode()` (unsafe since charger in Core 1), handler send `RelayCommand` to `gCommandQueue`:

```
BTN1 (TOUCH_BTN_PIN = GPIO 4) — core 0 handler:
  ├─ heldMs ≤ 600ms:  {CmdType::RESET_ENERGY} →
xQueueSend(gCommandQueue)
  │                 → Core 1: pzem.resetEnergy() + pushAlert()
  ├─ heldMs ≥ 2000ms: read snap.smartMode from gSharedState
  │                 → {CmdType::SMART_MODE, !curSmart} → xQueueSend()
  │                 → Core 1: charger.setSmartMode()
  └─ dead zone:       skip
```

BTN2 (TOUCH_BTN2_PIN = GPIO 33) — core 0 handler:

├─ heldMs ≤ 800ms: gUi.toggleDisplay() — GPIO32 (TFT BL) — local Core 0

├─ heldMs ≥ 3000ms: startConfigPortal → gConfigMode=true — local Core 0

└─ dead zone: skip

2.3.4. Two phase DS18B20 mechanism (SafetyMonitor - Core 1)

Phase 1 — Send measure command (when SENSOR_READ_MS = 1000ms):

_ds.requestTemperatures() ← return IMMEDIATELY, non-blocking

_tempRequestedMs = now

_awaitingTemp = true

[task continue the 50ms cycle: read PZEM, process queue, update charger...]

Phase 2 — Read result (after _awaitingTemp && 800ms has passed):

t0 = _ds.getTempCByIndex(0) ← Sensor index 0 (socket)

t1 = _ds.getTempCByIndex(1) ← Sensor index 1 (SSR)

→ check _isValidTemp()

→ calculate maxT from valid sensors

→ _status.overheat = (maxT > 60.0f) and no sensorFault

→ _status.fireRisk = (maxT > 75.0f) and no sensorFault

_awaitingTemp = false

2.3.5. MQTT dataflow

Telemetry payload (publishTelemetry — Core 0, every 1 seconds from `SharedState` snapshot):

```
{
  "voltage": 220.54,
  "current": 1.423,
  "power": 313.58,
  "energy_wh": 1024.3,
```

```
  "temp1": 45.1,
  "temp2": 38.6,
  "relay_state": "ON",
  "charge_mode": "ON"
}
```

Control payload (Dashboard → ESP32, through `TOPIC_CONTROL`):

```
{ "relay": "ON" }
{ "relay": "OFF" }
{ "charge_mode": "ON" }
{ "charge_mode": "OFF" }
```

`mqttCallback()` parse by `indexOf()` then `xQueueSend(gCommandQueue, cmd, 0)` — send to queue Core 1.

ACK payload (Core 0 publish after 150ms, `TOPIC_ACK`):

```
{
  "command": "relay",
  "requested_value": "ON",
  "actual_value": "ON",
  "timestamp": 1234567890,
  "status": "confirmed"
}
```

ACK verifies by reading `gSharedState.relayOn` (updated by Core 1) through mutex.

Alert payload (Core 1 push vào `gAlertQueue`, Core 0 publish với `retain=true`):

```
{ "alert": "FIRE RISK >75C: relay locked until hard reset" }
```

2.3.6. Dashboard architecture(server.js + [app.js](app.js))

Socket.IO Events (from `server.js` and `app.js`):

Event	Route	Trigger	Process
`state_update`	Server → Browser	MQTT telemetry or alert	`updateMetrics()`, `updateStatus()`, `updateIndicators()`
`command_sent`	Server → Browser	After publish MQTT	`showAlert()`, `addLog()`
`command_acked`	Server → Browser	MQTT telemetry or alert	`updateMetrics()`, `updateStatus()`, `updateIndicators()`
`command_sent`	Server → Browser	After publish MQTT	`showAlert()`, `addLog()`
`command_acked`	Server → Browser	MQTT ACK receive	`showAlert()`, `addLog()`
`alert_received`	Server → Browser	MQTT alert receive	`showAlert()`, `addLog()`
`control_relay`	Browser → Server	User click	`mqttClient.publish(TOPICS.CONTROL, {"relay":"ON/OFF"})`
`toggle_smart_mode`	Browser → Server	User click	`mqttClient.publish(TOPICS.CONTROL, {"charge_mode":"ON/OFF"})`
`request_state`	Browser → Server	On load, every 30s	`socket.emit('state_update', deviceState)`
`command_history`	Server → Browser	On new connection	Send `commandHistory[]`

REST API (từ `server.js`):

Method	Endpoint	Description
GET	`/api/status`	Return current `deviceState` JSON
POST	`/api/control`	Receive `{command, value}`, send to `io.emit`
GET	`/api/history`	Command history, query param `?limit=N`, default 50
GET	`/api/command/:id`	Query command based on ID
GET	`/api/command/status/:id`	ACK state of a command based on ID

3. Deploy & Coding

3.1. Firmware for ESP32

3.1.1. `config.h`

All hardware variables, algorithm variables and config for FreeRTOS is defined once in `include/config.h` Ensure there is no magic number in `*.cpp`

```
// Safety thresholds
#define TEMP_ALERT_C    60.0f     // Overheat → relay OFF
(recoverable)
#define TEMP_FIRE_C     75.0f     // Fire risk → relay LOCKED
(permanent)

// Smart charging parameters
#define CHARGE_TIMEOUT_MS  (8UL * 60UL * 60UL * 1000UL)  // 8
giờ
#define FULL_CURRENT_A      0.10f    // Trickle threshold: 100mA
#define FULL_HOLD_MS       (5UL * 60UL * 1000UL)          // 5
phút liên tục
#define UNPLUG_CURRENT_A    0.01f
```

```
#define UNPLUG_FROM_A       0.05f
#define SPIKE_FACTOR        1.30f
#define ROLLING_WINDOW_SEC  300U     // 5 minutes window = 300
sample × 1s

// Button timing
#define BTN_DEBOUNCE_MS      50UL
#define BTN_SINGLE_MAX_MS   600UL
#define BTN_LONG_MIN_MS    2000UL
#define BTN2_SINGLE_MAX_MS  800UL
#define BTN2_PORTAL_MIN_MS 3000UL

// Timer intervals
#define SENSOR_READ_MS       1000UL
#define DISPLAY_REFRESH_MS    500UL
#define TELEMETRY_PUBLISH_MS 1000UL
#define WIFI_RETRY_MS        5000UL
#define MQTT_RETRY_MS        3000UL

// MQTT
#define TOPIC_TELEMETRY  "socket/telemetry"
#define TOPIC_CONTROL    "socket/control"
#define TOPIC_ALERT      "socket/alert"
#define TOPIC_ACK        "socket/control/ack"
#define MQTT_HOST
"413f71f93ddf45f4a270637f32f088fd.s1.eu.hivemq.cloud"
#define MQTT_PORT         8883
#define MQTT_USE_TLS      true
#define MQTT_CLIENTID    "iot-smart-socket-esp32"

// ── FreeRTOS Task Configuration
──────────────────────────────────────────────────────────
#define RTOS_CHARGE_CORE       1   // PRO core: safety-critical
#define RTOS_NETCOMMS_CORE     0   // APP core: networking +
display
#define RTOS_CHARGE_PRIO       3   // High priority
```

```
#define RTOS_NETCOMMS_PRIO     1   // Low priority
#define RTOS_CHARGE_STACK      8192
#define RTOS_NETCOMMS_STACK    10240
#define RTOS_CMD_QUEUE_DEPTH   6
#define RTOS_ALERT_QUEUE_DEPTH 12
#define CHARGE_TASK_PERIOD_MS  50  // 20 Hz cycle for
taskChargeSafety
```

3.1.2. `SharedState.h` — Shared data between 2 cores

```
// Fixed-size alert: avoid heap allocation in task context
#define ALERT_TEXT_MAX 96
struct AlertMessage { char text[ALERT_TEXT_MAX]; };

// Command from Core 0 → Core 1
enum class CmdType : uint8_t { RELAY, SMART_MODE, RESET_ENERGY
};
struct RelayCommand { CmdType type; bool value; };

// Snapshot data: Core 1 write, Core 0 read (through
gStateMutex)
struct SharedState {
    PowerReading     power{};
    SafetyStatus     safety{};
    bool             relayOn       = false;
    bool             smartMode     = true;
    ChargeStopReason stopReason    = ChargeStopReason::NONE;
    bool             pzemFault     = true;
    bool             fireLocked    = false;
    // Core 0 write the following field:
    bool             wifiConnected = false;
    bool             mqttConnected = false;
    bool             configMode    = false;
    String           lastAlert     = "System boot";
};
```

```
extern SemaphoreHandle_t gStateMutex;
extern QueueHandle_t     gCommandQueue;
extern QueueHandle_t     gAlertQueue;
extern SharedState       gSharedState;
```

3.1.3. Smart Charging Algorithm - `ChargeManager.cpp`

```
void ChargeManager::_pushCurrent(float current) {
    if (_winCount < ROLLING_WINDOW_SEC) {
        // Giai đoạn warm-up: điền buffer từ đầu
        _window[_winIndex] = current;
        _winSum += current;
        _winCount++;
        _winIndex = (_winIndex + 1U) % ROLLING_WINDOW_SEC;
        return;
    }
    // Buffer đầy: sliding window -- bỏ phần tử cũ nhất
    _winSum -= _window[_winIndex];
    _window[_winIndex] = current;
    _winSum += current;
    _winIndex = (_winIndex + 1U) % ROLLING_WINDOW_SEC;
}
```

3.1.4. `taskChargeSafety` - Core 1, priority 3

```
static void taskChargeSafety(void* /*param*/) {
    // Core-1-local objects -- never access from Core 0
    PZEM004Tv30   pzem(PZEM_SERIAL, PZEM_RX_PIN, PZEM_TX_PIN);
    ChargeManager charger;
    SafetyMonitor safety;

    charger.begin();
    safety.begin();

    bool             localFireLocked = false;
    PowerReading     localPower{};
```

```
    ChargeStopReason prevStopReason  = ChargeStopReason::NONE;

    // Helper: push alert into queue (non-blocking)
    auto pushAlert = [](const char *msg) {
        AlertMessage am{};
        strncpy(am.text, msg, ALERT_TEXT_MAX - 1);
        xQueueSend(gAlertQueue, &am, 0);  // drop if queue full
    };

    TickType_t xLastWake = xTaskGetTickCount();

    for (;;) {
        // Ensure fixed cycle 50ms -- non-drifting
        vTaskDelayUntil(&xLastWake,
pdMS_TO_TICKS(CHARGE_TASK_PERIOD_MS));

        // 1. Drain command queue from Core 0
        RelayCommand cmd{};
        while (xQueueReceive(gCommandQueue, &cmd, 0) == pdTRUE)
{
            switch (cmd.type) {
                case CmdType::RELAY:
                    charger.commandRelay(cmd.value,
localFireLocked); break;
                case CmdType::SMART_MODE:
                    charger.setSmartMode(cmd.value); break;
                case CmdType::RESET_ENERGY:
                    pzem.resetEnergy(); pushAlert("Energy
counter reset"); break;
            }
        }

        // 2. Read PZEM @1Hz
        // ... (millis gate 1000ms)

        // 3. Safety monitor
```

```
        safety.setPeripheralFault(localPzemFault);
        safety.update();

        if (s.overheat)
charger.forceRelayOff(ChargeStopReason::SAFETY);
        if (s.fireRisk && !localFireLocked) {
            localFireLocked = true;  // PERMANENT -- no code
reset
            charger.forceRelayOff(ChargeStopReason::SAFETY);
            pushAlert("FIRE RISK >75C: relay locked until hard
reset");
        }

        // 4. Charge algorithm
        charger.update(localPower, localFireLocked);

        // 5. Write shared state (mutex maximum 10ms)
        if (xSemaphoreTake(gStateMutex, pdMS_TO_TICKS(10)) ==
pdTRUE) {
            gSharedState.power      = localPower;
            gSharedState.safety     = safety.status();
            gSharedState.relayOn    = charger.relayOn();
            gSharedState.smartMode  = charger.smartMode();
            gSharedState.stopReason = charger.lastStopReason();
            gSharedState.fireLocked = localFireLocked;
            xSemaphoreGive(gStateMutex);
        }
    }
}
```

3.1.5. `taskNetComms` — Core 0, Priority 1

```
static void taskNetComms(void* /*param*/) {
    // Core-0-local: WiFi, MQTT, DisplayUI
    gUi.begin();   // TFT init -- SPI init only from Core 0
```

```
    // Connect WiFi (prioritize saved credentials)
    // ... WiFi.begin() + wm setup

    gMqtt.setServer(MQTT_HOST, MQTT_PORT);
    gMqtt.setCallback(mqttCallback);  // mqttCallback call
xQueueSend

    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(10));

        // BTN1/BTN2 polling + FSM → xQueueSend(gCommandQueue,
...)
        // WiFi management + WiFiManager portal
        // MQTT connect + mqtt.loop()

        // Drain alert queue from Core 1
        AlertMessage am{};
        while (xQueueReceive(gAlertQueue, &am, 0) == pdTRUE) {
            netPublishAlert(String(am.text));
        }

        // Read snapshot safety
        SharedState snap{};
        if (xSemaphoreTake(gStateMutex, pdMS_TO_TICKS(5)) ==
pdTRUE) {
            snap = gSharedState;
            xSemaphoreGive(gStateMutex);
        }

        gUi.update(snap);          // DisplayUI receive snapshot
        publishTelemetry(snap);    // JSON from snapshot
    }
}
```

3.1.6. `setup()` & `loop()` — Initialize RTOS

```
void setup() {
    Serial.begin(115200);

    // Init GPIO before start tasks
    pinMode(TOUCH_BTN_PIN,  INPUT);
    pinMode(TOUCH_BTN2_PIN, INPUT);
    // ... other pin

    WiFiStorage::initSPIFS();

    // Create RTOS primitives
    gStateMutex   = xSemaphoreCreateMutex();
    gCommandQueue = xQueueCreate(RTOS_CMD_QUEUE_DEPTH,
sizeof(RelayCommand));
    gAlertQueue   = xQueueCreate(RTOS_ALERT_QUEUE_DEPTH,
sizeof(AlertMessage));

    // Check success initialize
    configASSERT(gStateMutex);
    configASSERT(gCommandQueue);
    configASSERT(gAlertQueue);

    // Spawn tasks -- pinned to cores
    xTaskCreatePinnedToCore(
        taskChargeSafety, "ChargeSafety",
        RTOS_CHARGE_STACK, nullptr,
        RTOS_CHARGE_PRIO, &hChargeSafety,
        RTOS_CHARGE_CORE    // Core 1
    );

    xTaskCreatePinnedToCore(
        taskNetComms, "NetComms",
        RTOS_NETCOMMS_STACK, nullptr,
        RTOS_NETCOMMS_PRIO, &hNetComms,
        RTOS_NETCOMMS_CORE  // Core 0
    );
```

```
}
```

3.1.7. `DisplayUI` — Refactor receive SharedState Snapshot

```
// DisplayUI.h -- constructor default, no reference
class DisplayUI {
public:
    DisplayUI() = default;
    void begin();
    void update(const SharedState &snap);   // instead of many
parameters
    void toggleDisplay();
    bool isDisplayOn() const;
private:
    TFT_eSPI _tft{};
};

// DisplayUI.cpp -- read from snap instead of directly calling
methods
void DisplayUI::update(const SharedState &snap) {
    if (!_displayOn) return;
    // ... render from snap.power, snap.safety, snap.relayOn,
snap.lastAlert
}
```

3.1.8. `SafetyMonitor` — LED and Buzzer Logic (Core 1)

```
void SafetyMonitor::_updateLedsAndBuzzer() {
    const bool hasFault = _status.sensorFault ||
_peripheralFault;
    const bool redOn = hasFault || _status.overheat ||
_status.fireRisk;

    digitalWrite(LED_RED_PIN, redOn ? HIGH : LOW);   // Owned by
core 1
    digitalWrite(BUZZER_PIN, _status.fireRisk ? HIGH : LOW);  //
```

```
Owned by core 1
    // LED_GREEN_PIN managed by taskNetComms on Core 0
}
```

3.1.9. WiFi Management (Core 0)

```
WiFiCredentials creds{"", ""};
WiFi.mode(WIFI_STA);
if (WiFiStorage::loadWiFiCredentials(creds) &&
strlen(creds.ssid) > 0) {
    WiFi.begin(creds.ssid, creds.password);  // Prioritize saved
credentials
} else {
    WiFi.begin(WIFI_SSID, WIFI_PASS);        // Fallback
hardcoded
}
```

3.1.10. `main.cpp` — Command ACK mechanism (Through Shared State)

```
// Pending commands verify (in taskNetComms, Core 0):
SharedState snap{};
if (xSemaphoreTake(gStateMutex, pdMS_TO_TICKS(5)) == pdTRUE) {
    snap = gSharedState;  // Core 1 update through
relayOn/smartMode
    xSemaphoreGive(gStateMutex);
}

if (command == "relay") {
    actual   = snap.relayOn  ? "ON" : "OFF";  // Read from
snapshot
    verified = (actual == requested);
}
```

3.2. Code structure(Modular Design)

File	Role	Size
include/config.h	Global variable (GPIO, threshold, timer)	2.6 KB
include/SharedState.h	Struct IPC between 2 core: SharedState, RelayCommand, AlertMessage, extern RTOS handles	1.8 KB
include/ChargeManager.h	Interface + struct PowerReading, enum ChargeStopReason	1.3 KB
include/SafetyMonitor.h	Interface + struct SafetyStatus	1.0 KB
include/DisplayUI.h	Interface	0.8 KB
include/WiFiStorage.h	Header-only static class	3.5 KB
src/main.cpp	RTOS: setup(), loop(), 2 tasks, MQTT callback, WiFi helpers	12.0 KB
src/ChargeManager.cpp	Charging algorithm and circular buffer	3.3 KB
src/SafetyMonitor.cpp	S18B20 async + LED/buzzer	1.9 KB
src/DisplayUI.cpp	TFT layout	3.6 KB
web-dashboard/server.js	Node.js: Express + MQTT + Socket.IO	9.5 KB
web-dashboard/public/js/app.js	Browser: Socket.IO client + DOM	7.1 KB

4. Evaluation and Testing

The test cases below are built based on the behavior defined in the code, including scenarios related to the multi-core RTOS

a) Group 1 - Power Measurement (PZEM-004T)

ID	Scenario	Conditions	Expected Results	Validation Method
TC-01	Normal power measurement	PZEM connected, with load	LCD & MQTT telemetry have no "null" or "--"	Serial Monitor + Dashboard
TC-02	PZEM disconnected	PZEM connected, with load	Alert to gAlertQueue -> Core 0 publishes MQTT, LCD shows "--", Red LED ON	Serial Monitor
TC-03	PZEM recovered	Reconnect UART after TC-02	localPzemFault =false, LCD displays values, Red LED OFF	LCD
TC-04	Reset Wh using BTN1	Press BTN1 < 600ms	CmdType::RESET_ENERGY → queue → Core 1: pzem.resetEnergy() + pushAlert	LCD Energy = 0 in ~50ms

b) Group 2 — Thermal Safety (SafetyMonitor — Core 1)

ID	Scenario	Conditions	Expected Results	Validation Method
TC-05	DS18B20 offline	Disconnect sensor wire	sensorFault=true, LCD shows "--", Red LED ON	Serial Monitor
TC-06	Normal temperature	T < 60°C	Red LED OFF, relay unaffected	Physical LED

TC-07	Overheat > 60°C	Heat the sensor	overheat=true, relay OFF, Red LED ON, alert via gAlertQueue	Relay LED OFF
TC-08	Fire Lock > 75°C	Continue heating	localFireLocked=true on Core 1, relay LOCK, buzzer ON	Relay does not turn back on
TC-09	ON command after Fire Lock	Publish {"relay":"ON"} after TC-08	Core 1 receives cmd, commandRelay(true, true) returns immediately	ACK "failed"

c) Group 3 — Smart Charging (ChargeManager — Core 1)

ID	Scenario	Conditions	Expected Results	Validation Method
TC-10	Battery full detection	Smart Mode ON, current < 0.10A	After 5 mins, relay OFF, gSharedState.stopReason=FULL	Dashboard + Relay
TC-11	Reset Full timer	Current drops < 0.10A then rises	_lowCurrentStartMs = 0, no early cutoff	Relay remains ON
TC-12	Unplug detection	Current drops from > 0.05A to < 0.01A	After 2 samples (~100ms with 50ms cycle), relay OFF	Dashboard
TC-13	Spike detection	Current rises > 1.30× rolling avg	Relay OFF, stopReason=SPIKE, alert via queue	Serial Monitor
TC-14	8-hour timeout	Relay ON continuously	After 8 hours, relay OFF, stopReason=TIMEOUT	Log
TC-15	Manual Mode bypass	Smart Mode OFF	No Full/Unplug/Spike, only Timeout	Relay remains ON when current < 0.10A

d) Group 4 — MQTT & Remote Control

ID	Scenario	Conditions	Expected Results	Validation Method
TC-16	Relay ON from MQTT	Publish {"relay":"ON"}	mqttCallback → gCommandQueue → Core 1 → relay ON, ACK in ~150ms	Dashboard + ACK payload
TC-17	Relay OFF from MQTT	Publish {"relay":"OFF"}	Relay OFF, ACK confirmed	Dashboard
TC-18	Smart mode from MQTT	Publish {"charge_mode":"ON"}	Core 1 setSmartMode, gSharedState updated, LCD, ACK	LCD
TC-19	Wrong command format	Publish relay=ON (wrong format)	indexOf() does not match, does not enqueue, no crash	Serial Monitor: no relay log
TC-20	ACK verify from SharedState	After any relay command	Core 0 reads snap.relayOn from mutex, full JSON ACK	MQTT client

e) Group 5 — RTOS Multi-Core & Connectivity

ID	Scenario	Conditions	Expected Results	Validation Method
TC-21	WiFi loss	Turn off router	Core 0: retry; Core 1: operates normally, safety active	Serial log "[Core1]" continues
TC-22	Safety during WiFi loss	Network loss when T > 60°C	Core 1 cuts relay immediately, gAlertQueue stores alert waiting for	Physical relay OFF

			WiFi	
TC-23	WiFi recovery	Turn router back on	Core 0 reconnects, drains gAlertQueue, publishes delayed alerts	LCD WiFi/MQTT → CONNECTED
TC-24	Core isolation	Flood MQTT commands simultaneously	Core 1 does not stall — gCommandQueue buffers, Core 1 drains sequentially	Relay still responds correctly
TC-25	Hold BTN2 for 3s	Hold BTN2 ≥ 3000ms	Core 0 WiFi portal; Core 1 continues reading PZEM + safety	"SmartSocket-Setup" + relay remains active

5. Innovation & Scalability

5.1. Technical innovations

- **Reliable remote control verification:** The system uses a robust method to ensure remote commands (like turning the power on or off via the dashboard) are actually executed. Instead of simply assuming a command worked, the communication core queues the command, the safety core executes it, and then the communication core independently verifies the hardware's new state before sending a confirmation back to the user. This prevents the dashboard from showing an incorrect status if a mechanical failure occurs.
- **Complete system isolation (Dual-Core architecture):** A major advancement in this design is the strict separation of duties between the two processing cores. One core is exclusively dedicated to safety monitoring, operating on a rigid, uninterrupted schedule. It has sole control over critical hardware like sensors and the power relay. The second core handles all communication, network reconnections, and user interfaces. This guarantees that delays caused by poor network connections or screen updates never compromise the system's ability to detect hazards and cut power instantly.

- **Secure internal data handling:** The method used to pass information between the two processors is designed to prevent data corruption and memory issues. Instead of changing shared information directly, the system uses synchronized queues for commands and securely locks shared data when it's being read or updated. This ensures smooth operation even when both cores are highly active.
- **Advanced smart charging logic:** The decision to cut power isn't based on a simple timer. The system intelligently monitors four distinct conditions simultaneously:
    + **Full charge:** Detecting when the current drops to a very low level for a sustained period, which is optimal for the health of lithium batteries.
    + **Unplugged device:** Rapidly detecting a sudden drop in current to near zero, confirmed over multiple, highly precise readings.
    + **Current spikes:** Identifying sudden, dangerous surges in current compared to a recent historical average.
    + **Maximum time limit**: A hard, fixed time limit serving as an ultimate fail-safe.
- **Uninterrupted Safety During Setup:** Even when a user is actively configuring the device's network settings through a web portal, the system's safety core continues to monitor temperatures and manage the relay without any pause or delay.

5.2. Future scalability

- **Historical data logging:** The hardware infrastructure is already in place to support saving long-term data. Future updates can easily enable writing detailed usage and sensor logs directly to an SD card for later analysis.
- **Seamless remote updates:** The system is built to support receiving software updates over the air. Because of the dual-core design, future updates could be downloaded and prepared by the communication core while the safety core continues to actively monitor and protect the connected devices.
- **Production-Grade reliability:** The system is primed to integrate a hardware watchdog timer. This acts as an ultimate safety net that can automatically force a full system reboot if the critical safety monitoring task ever freezes or fails to respond within a tight timeframe.
- **Managing multiple devices:** The communication structure is designed so that the central server can easily be upgraded to manage and identify numerous smart

sockets simultaneously, laying the groundwork for a broader smart home or facility-wide deployment.

6. Conclusion

The Ender Socket project successfully fulfills its primary objective: transforming a standard electrical socket into an intelligent, highly secure IoT device capable of managing high-capacity lithium battery charging. By migrating from a traditional single-loop design to a robust Dual-Core Real-Time Operating System (RTOS) architecture, the system achieves commercial-grade reliability.

All established functional requirements (FR-01 through FR-10) have been fully implemented and validated against the system's strict real-time and safety constraints.

Requirement	**Fulfillment**	**Architectural Note**
Power & Environmental monitoring	Fully implemented	Handled exclusively by the isolated safety core. This guarantees continuous, real-time tracking of voltage, current, and dual-zone temperatures without any interference from network latency.
Smart charging & Hardware safety	Fully implemented	All four multi-criteria shutoff rules (Full, Unplug, Spike, Timeout) and the irreversible Fire Lock mechanism execute with strict latency guarantees, acting as an absolute fail-safe.
Local & Remote actuation	Fully implemented	Physical button presses and remote MQTT commands utilize a secure, inter-core communication queue, ensuring reliable hardware triggering and accurate visual feedback.

Connectivity & Telemetry	Fully implemented	The WiFi Captive Portal, real-time data broadcasting, and cloud reconnections operate smoothly in a non-blocking manner entirely on the secondary communication core.
User interface & Display	Fully implemented	The physical LCD screen updates seamlessly using shared system states, providing the user with live data without halting or delaying any underlying safety checks.

Overall Assessment: The system excels by ensuring that network instability—a common point of failure in standard IoT devices—has zero impact on hardware safety. The prototype successfully balances cost-effective hardware with a sophisticated, failsafe software architecture.